

! Note

You are reading the documentation for a pre-release version. The latest stable version is [v6.0](#)

SD/SDIO/MMC Driver

[\[中文\]](#)

Overview

The SD/SDIO/MMC driver supports SD memory, SDIO cards, and eMMC chips. This is a protocol layer driver ([sdmmc/include/sdmmc_cmd.h](#)) which can work together with:

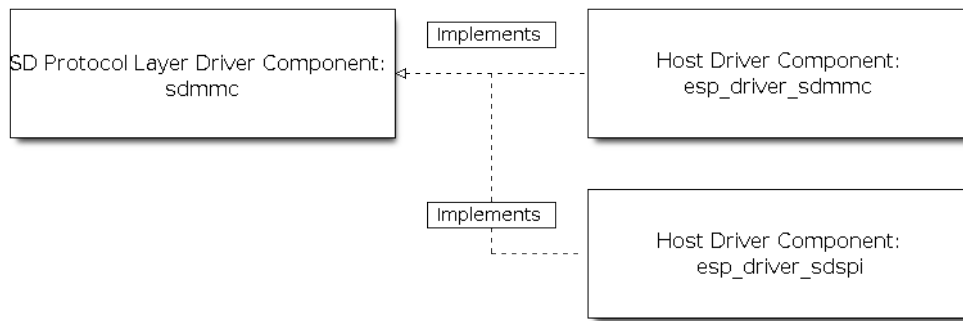
- SDMMC host driver ([esp_driver_sdmmc/include/driver/sdmmc_host.h](#)), see [SDMMC Host API](#) for more details.
- SDSPI host driver ([esp_driver_sdspi/include/driver/sdspi_host.h](#)), see [SD SPI Host API](#) for more details.

Protocol Layer vs Host Layer

The SDMMC protocol layer described in this document handles the specifics of the SD protocol, such as the card initialization flow and various data transfer command flows. The protocol layer works with the host via the `sdmmc_host_t` structure. This structure contains pointers to various functions of the host.

Host layer driver(s) implement the protocol layer driver by supporting these functions:

- Sending commands to slave devices
- Sending and receiving data
- Handling error conditions within the bus



SD Host Side Component Architecture

Application Examples

- [storage/sd_card/sdmmc](#) demonstrates how to operate an SD card formatted with the FatFS file system via the SDMMC interface.
- [storage/emmc](#) demonstrates how to operate an eMMC chip formatted with the FatFS file system via the SDMMC interface.
- [storage/sd_card/sdspi](#) demonstrates how to operate an SD card formatted with the FatFS file system via the SPI interface.

Protocol Layer API

The protocol layer is given the `sdmmc_host_t` structure. This structure describes the SD/MMC host driver, lists its capabilities, and provides pointers to functions for the implementation driver. The protocol layer stores card-specific information in the `sdmmc_card_t` structure. When sending commands to the SD/MMC host driver, the protocol layer uses the `sdmmc_command_t` structure to describe the command, arguments, expected return values, and data to transfer if there is any.

Using API with SD Memory Cards

- To initialize the SDMMC host, call the host driver functions, e.g., `sdmmc_host_init()`, `sdmmc_host_init_slot()` .
- To initialize the SDSPI host, call the host driver functions, e.g., `sdspi_host_init()`, `sdspi_host_init_slot()` .
- To initialize the card, call `sdmmc_card_init()` and pass to it the parameters `host` - the host driver information, and `card` - a pointer to the structure `sdmmc_card_t` which will be filled with information about the card when the function completes.
- To read and write sectors of the card, use `sdmmc_read_sectors()` and `sdmmc_write_sectors()` respectively and pass to it the parameter `card` - a pointer to the card information structure.

- If the card is not used anymore, call the host driver function to disable the host peripheral and free the resources allocated by the driver (`sdmmc_host_deinit` for SDMMC or `sdspi_host_deinit` for SDSPI).

Using API with eMMC Chips

From the protocol layer's perspective, eMMC memory chips behave exactly like SD memory cards. Even though eMMCs are chips and do not have a card form factor, the terminology for SD cards can still be applied to eMMC due to the similarity of the protocol (`sdmmc_card_t`, `sdmmc_card_init`). Note that eMMC chips cannot be used over SPI, which makes them incompatible with the SD SPI host driver.

To initialize eMMC memory and perform read/write operations, follow the steps listed for SD cards in the previous section.

Using API with SDIO Cards

Initialization and the probing process are the same as with SD memory cards. The only difference is in data transfer commands in SDIO mode.

During the card initialization and probing, performed with `sdmmc_card_init()`, the driver only configures the following registers of the IO card:

1. The IO portion of the card is reset by setting RES bit in the I/O Abort (0x06) register.
2. If 4-line mode is enabled in host and slot configuration, the driver attempts to set the Bus width field in the Bus Interface Control (0x07) register. If setting the field is successful, which means that the slave supports 4-line mode, the host is also switched to 4-line mode.
3. If high-speed mode is enabled in the host configuration, the SHS bit is set in the High Speed (0x13) register.

In particular, the driver does not set any bits in (1) I/O Enable and Int Enable registers, (2) I/O block sizes, etc. Applications can set them by calling `sdmmc_io_write_byte()`.

For card configuration and data transfer, choose the pair of functions relevant to your case from the table below.

Action	Read Function
Read and write a single byte using IO_RW_DIRECT (CMD52)	<code>sdmmc_io_read_byte()</code>
Read and write multiple bytes using IO_RW_EXTENDED (CMD53) in byte mode	<code>sdmmc_io_read_bytes()</code>

Action	Read Function
Read and write blocks of data using IO_RW_EXTENDED (CMD53) in block mode	<code>sdmmc_io_read_block</code>

SDIO interrupts can be enabled by the application using the function `sdmmc_io_enable_int()`. When using SDIO in 1-line mode, the D1 line also needs to be connected to use SDIO interrupts.

If you want the application to wait until the SDIO interrupt occurs, use `sdmmc_io_wait_int()`.

Combo (Memory + IO) Cards

The driver does not support SD combo cards. Combo cards are treated as IO cards.

Thread Safety

Most applications need to use the protocol layer only in one task. For this reason, the protocol layer does not implement any kind of locking on the `sdmmc_card_t` structure, or when accessing SDMMC or SD SPI host drivers. Such locking is usually implemented on a higher layer, e.g., in the filesystem driver.

API Reference

Header File

- [components/sdmmc/include/sdmmc_cmd.h](#)
- This header file can be included with:

```
#include "sdmmc_cmd.h"
```

- This header file is a part of the API provided by the `sdmmc` component. To declare that your component depends on `sdmmc`, add the following to your CMakeLists.txt:

```
REQUIRES sdmmc
```

or

```
PRIV_REQUIRES sdmmc
```

Functions

```
esp_err_t sdmmc_card_init(const sdmmc_host_t *host, sdmmc_card_t *out_card)
```

Probe and initialize SD/MMC card using given host

- Parameters:**
- **host** -- pointer to structure defining host controller
 - **out_card** -- pointer to structure which will receive information about the card when the function completes
- Returns:**
- ESP_OK on success
 - One of the error codes from SDMMC host controller

```
void sdmmc_card_print_info(FILE *stream, const sdmmc_card_t *card)
```

Print information about the card to a stream.

- Parameters:**
- **stream** -- stream obtained using fopen or fdopen
 - **card** -- card information structure initialized using sdmmc_card_init

```
esp_err_t sdmmc_get_status(sdmmc_card_t *card)
```

Get status of SD/MMC card

- Parameters:**
- **card** -- pointer to card information structure previously initialized using sdmmc_card_init
- Returns:**
- ESP_OK on success
 - One of the error codes from SDMMC host controller

```
esp_err_t sdmmc_write_sectors(sdmmc_card_t *card, const void *src, size_t start_sector, size_t sector_count)
```

Write given number of sectors to SD/MMC card

- Parameters:**
- **card** -- pointer to card information structure previously initialized using `sdmmc_card_init`
 - **src** -- pointer to data buffer to read data from; data size must be equal to `sector_count * card->csd.sector_size`
 - **start_sector** -- sector where to start writing
 - **sector_count** -- number of sectors to write

- Returns:**
- ESP_OK on success or `sector_count` equal to 0
 - One of the error codes from SDMMC host controller

`esp_err_t sdmmc_read_sectors(sdmmc_card_t *card, void *dst, size_t start_sector, size_t sector_count)`

Read given number of sectors from the SD/MMC card

- Parameters:**
- **card** -- pointer to card information structure previously initialized using `sdmmc_card_init`
 - **dst** -- pointer to data buffer to write into; buffer size must be at least `sector_count * card->csd.sector_size`
 - **start_sector** -- sector where to start reading
 - **sector_count** -- number of sectors to read

- Returns:**
- ESP_OK on success or `sector_count` equal to 0
 - One of the error codes from SDMMC host controller

`esp_err_t sdmmc_erase_sectors(sdmmc_card_t *card, size_t start_sector, size_t sector_count, sdmmc_erase_arg_t arg)`

Erase given number of sectors from the SD/MMC card

! Note

When `sdmmc_erase_sectors` used with cards in SDSPI mode, it was observed that card requires re-init after erase operation.

- Parameters:**
- **card** -- pointer to card information structure previously initialized using `sdmmc_card_init`
 - **start_sector** -- sector where to start erase
 - **sector_count** -- number of sectors to erase
 - **arg** -- erase command (CMD38) argument

- Returns:**
- ESP_OK on success or `sector_count` equal to 0
 - One of the error codes from SDMMC host controller

`esp_err_t sdmmc_can_discard(sdmmc_card_t *card)`

Check if SD/MMC card supports discard

Parameters: `card` -- pointer to card information structure previously initialized using `sdmmc_card_init`

Returns:

- ESP_OK if supported by the card/device
- ESP_FAIL if not supported by the card/device

`esp_err_t sdmmc_can_trim(sdmmc_card_t *card)`

Check if SD/MMC card supports trim

Parameters: `card` -- pointer to card information structure previously initialized using `sdmmc_card_init`

Returns:

- ESP_OK if supported by the card/device
- ESP_FAIL if not supported by the card/device

`esp_err_t sdmmc_mmc_can_sanitize(sdmmc_card_t *card)`

Check if SD/MMC card supports sanitize

Parameters: `card` -- pointer to card information structure previously initialized using `sdmmc_card_init`

Returns:

- ESP_OK if supported by the card/device
- ESP_FAIL if not supported by the card/device

`esp_err_t sdmmc_mmc_sanitize(sdmmc_card_t *card, uint32_t timeout_ms)`

Sanitize the data that was unmapped by a Discard command

Note

Discard command has to precede sanitize operation. To discard, use `MMC_DICARD_ARG` with `sdmmc_erase_sectors` argument

Parameters:

- `card` -- pointer to card information structure previously initialized using `sdmmc_card_init`
- `timeout_ms` -- timeout value in milliseconds required to sanitize the selected range of sectors.

Returns:

- ESP_OK on success
- One of the error codes from SDMMC host controller

esp_err_t sdmmc_full_erase(sdmmc_card_t *card)

Erase complete SD/MMC card

Parameters: **card** -- pointer to card information structure previously initialized using sdmmc_card_init

Returns:

- ESP_OK on success
- One of the error codes from SDMMC host controller

esp_err_t sdmmc_io_read_byte(sdmmc_card_t *card, uint32_t function, uint32_t reg, uint8_t *out_byte)

Read one byte from an SDIO card using IO_RW_DIRECT (CMD52)

Parameters:

- **card** -- pointer to card information structure previously initialized using sdmmc_card_init
- **function** -- IO function number
- **reg** -- byte address within IO function
- **out_byte** -- [out] output, receives the value read from the card

Returns:

- ESP_OK on success
- One of the error codes from SDMMC host controller

esp_err_t sdmmc_io_write_byte(sdmmc_card_t *card, uint32_t function, uint32_t reg, uint8_t in_byte, uint8_t *out_byte)

Write one byte to an SDIO card using IO_RW_DIRECT (CMD52)

Parameters:

- **card** -- pointer to card information structure previously initialized using sdmmc_card_init
- **function** -- IO function number
- **reg** -- byte address within IO function
- **in_byte** -- value to be written
- **out_byte** -- [out] if not NULL, receives new byte value read from the card (read-after-write).

Returns:

- ESP_OK on success
- One of the error codes from SDMMC host controller

esp_err_t sdmmc_io_read_bytes(sdmmc_card_t *card, uint32_t function, uint32_t addr, void *dst, size_t size)

Read multiple bytes from an SDIO card using IO_RW_EXTENDED (CMD53)

This function performs read operation using CMD53 in byte mode. For block mode, see `sdmmc_io_read_blocks`.

By default OP Code is set (incrementing address). To send CMD53 without this bit, OR the argument `addr` with `SDMMC_IO_FIXED_ADDR`.

- Parameters:**
- **card** -- pointer to card information structure previously initialized using `sdmmc_card_init`
 - **function** -- IO function number
 - **addr** -- byte address within IO function where reading starts
 - **dst** -- buffer which receives the data read from card. Aligned to 4 byte boundary unless `SDMMC_HOST_FLAG_ALLOC_ALIGNED_BUF` flag is set when calling `sdmmc_card_init`. The flag is mandatory when the buffer is behind the cache.
 - **size** -- number of bytes to read, 1 to 512.

- Returns:**
- ESP_OK on success
 - ESP_ERR_INVALID_SIZE if size exceeds 512 bytes
 - One of the error codes from SDMMC host controller

```
esp_err_t sdmmc_io_write_bytes(sdmmc_card_t *card, uint32_t function, uint32_t addr, const void *src, size_t size)
```

Write multiple bytes to an SDIO card using IO_RW_EXTENDED (CMD53)

This function performs write operation using CMD53 in byte mode. For block mode, see `sdmmc_io_write_blocks`.

By default OP Code is set (incrementing address). To send CMD53 without this bit, OR the argument `addr` with `SDMMC_IO_FIXED_ADDR`.

- Parameters:**
- **card** -- pointer to card information structure previously initialized using `sdmmc_card_init`
 - **function** -- IO function number
 - **addr** -- byte address within IO function where writing starts
 - **src** -- data to be written. Aligned to 4 byte boundary unless `SDMMC_HOST_FLAG_ALLOC_ALIGNED_BUF` flag is set when calling `sdmmc_card_init`. The flag is mandatory when the buffer is behind the cache.
 - **size** -- number of bytes to write, 1 to 512.

- Returns:**
- ESP_OK on success
 - ESP_ERR_INVALID_SIZE if size exceeds 512 bytes
 - One of the error codes from SDMMC host controller

`esp_err_t sdmmc_io_read_blocks(sdmmc_card_t *card, uint32_t function, uint32_t addr, void *dst, size_t size)`

Read blocks of data from an SDIO card using IO_RW_EXTENDED (CMD53)

This function performs read operation using CMD53 in block mode. For byte mode, see `sdmmc_io_read_bytes`.

By default OP Code is set (incrementing address). To send CMD53 without this bit, OR the argument `addr` with `SDMMC_IO_FIXED_ADDR`.

- Parameters:**
- **card** -- pointer to card information structure previously initialized using `sdmmc_card_init`
 - **function** -- IO function number
 - **addr** -- byte address within IO function where writing starts
 - **dst** -- buffer which receives the data read from card. Aligned to 4 byte boundary, and also cache line size if the buffer is behind the cache.
 - **size** -- number of bytes to read, must be divisible by the card block size.

- Returns:**
- ESP_OK on success
 - ESP_ERR_INVALID_SIZE if size is not divisible by 512 bytes
 - One of the error codes from SDMMC host controller

`esp_err_t sdmmc_io_write_blocks(sdmmc_card_t *card, uint32_t function, uint32_t addr, const void *src, size_t size)`

Write blocks of data to an SDIO card using IO_RW_EXTENDED (CMD53)

This function performs write operation using CMD53 in block mode. For byte mode, see `sdmmc_io_write_bytes`.

By default OP Code is set (incrementing address). To send CMD53 without this bit, OR the argument `addr` with `SDMMC_IO_FIXED_ADDR`.

- Parameters:**
- **card** -- pointer to card information structure previously initialized using `sdmmc_card_init`
 - **function** -- IO function number
 - **addr** -- byte address within IO function where writing starts
 - **src** -- data to be written. Aligned to 4 byte boundary, and also cache line size if the buffer is behind the cache.
 - **size** -- number of bytes to write, must be divisible by the card block size.

- Returns:**
- ESP_OK on success
 - ESP_ERR_INVALID_SIZE if size is not divisible by 512 bytes
 - One of the error codes from SDMMC host controller

esp_err_t sdmmc_io_enable_int(sdmmc_card_t *card)

Enable SDIO interrupt in the SDMMC host

Parameters: **card** -- pointer to card information structure previously initialized using sdmmc_card_init

Returns:

- ESP_OK on success
- ESP_ERR_NOT_SUPPORTED if the host controller does not support IO interrupts

esp_err_t sdmmc_io_wait_int(sdmmc_card_t *card, TickType_t timeout_ticks)

Block until an SDIO interrupt is received

Slave uses D1 line to signal interrupt condition to the host. This function can be used to wait for the interrupt.

Parameters:

- **card** -- pointer to card information structure previously initialized using sdmmc_card_init
- **timeout_ticks** -- time to wait for the interrupt, in RTOS ticks

Returns:

- ESP_OK if the interrupt is received
- ESP_ERR_NOT_SUPPORTED if the host controller does not support IO interrupts
- ESP_ERR_TIMEOUT if the interrupt does not happen in timeout_ticks

esp_err_t sdmmc_io_get_cis_data(sdmmc_card_t *card, uint8_t *out_buffer, size_t buffer_size, size_t *inout_cis_size)

Get the data of CIS region of an SDIO card.

You may provide a buffer not sufficient to store all the CIS data. In this case, this function stores as much data into your buffer as possible. Also, this function will try to get and return the size required for you.

Parameters:

- **card** -- pointer to card information structure previously initialized using sdmmc_card_init
- **out_buffer** -- Output buffer of the CIS data
- **buffer_size** -- Size of the buffer.
- **inout_cis_size** -- Mandatory, pointer to a size, input and output.
 - input: Limitation of maximum searching range, should be 0 or larger than buffer_size. The function searches for CIS_CODE_END until this range. Set to 0 to search infinitely.
 - output: The size required to store all the CIS data, if CIS_CODE_END is found.

Returns:

- ESP_OK: on success
- ESP_ERR_INVALID_RESPONSE: if the card does not (correctly) support CIS.
- ESP_ERR_INVALID_SIZE: CIS_CODE_END found, but buffer_size is less than required size, which is stored in the inout_cis_size then.
- ESP_ERR_NOT_FOUND: if the CIS_CODE_END not found. Increase input value of inout_cis_size or set it to 0, if you still want to search for the end; output value of inout_cis_size is invalid in this case.
- and other error code return from sdmmc_io_read_bytes

esp_err_t sdmmc_io_print_cis_info(uint8_t *buffer, size_t buffer_size, FILE *fp)

Parse and print the CIS information of an SDIO card.

! Note

Not all the CIS codes and all kinds of tuples are supported. If you see some unresolved code, you can add the parsing of these code in sdmmc_io.c and contribute to the IDF through the Github repository.

```
using sdmmc_card_init
```

Parameters:

- **buffer** -- Buffer to parse
- **buffer_size** -- Size of the buffer.
- **fp** -- File pointer to print to, set to NULL to print to stdout.

Returns:

- ESP_OK: on success
- ESP_ERR_NOT_SUPPORTED: if the value from the card is not supported to be parsed.
- ESP_ERR_INVALID_SIZE: if the CIS size fields are not correct.

Macros

SDMMC_IO_FIXED_ADDR

Call `sdmmc_io_read_bytes`, `sdmmc_io_write_bytes`, `sdmmc_io_read_blocks` or `sdmmc_io_write_blocks` APIs with address ORed by this flag to send CMD53 with OP Code clear (fixed address)

Header File

- [components/esp_driver_sdmmc/include/driver/sdmmc_types.h](#)
- This header file can be included with:

```
#include "driver/sdmmc_types.h"
```

- This header file is a part of the API provided by the `esp_driver_sdmmc` component. To declare that your component depends on `esp_driver_sdmmc`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_sdmmc
```

or

```
PRIV_REQUIRES esp_driver_sdmmc
```

Was this page helpful?

